

Formal Languages and Compilers

Prof. Breveglieri, Morzenti, Agosta

Written exam: laboratory question

30/06/2023

Time: 60 minutes. Textbooks and notes can be used. Pencil writing is allowed.

Important: Write your name on any additional sheet.

SURNAME (Cognome):

NAME (Nome):

Matricola:or Person Code:

Instructor: ☐ Prof. Breviglieri ☐ Prof. Morzenti ☐ Prof. Agosta

The laboratory question must be answered taking into account the implementation of the Acse compiler given with the exam text.

Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the Lance language with support for a **software-emulated division operator**.

The software-emulated division operator shall have the same functional behavior of regular division for positive dividend and divisors. However, *no DIV instructions* must be emitted by ACSE to translate it to assembly language. The operation may have undefined behavior if either or both dividend and divisor are negative — i.e. the implementation does not need to handle that case. Division by zero must be handled by computing a quotient of $2^{31} - 1$. The proposed implementation should perform proper constant folding. The code executed at compile time is not restricted in its use of division instructions. Therefore the C division operator may be freely used in the implementation, while the following functions are not allowed:

- `gen_div_instruction()`,
- `gen_divi_instruction()`,
- `handle_bin_numeric_op()` when the operator argument is `DIV`.

There are no constraints imposed regarding the run-time complexity of the software-emulated division.

The symbol associated to this new operator is a bracketed slash (`[/]`) instead of the familiar slash (`/`). Its precedence and associativity are the same as non-emulated standard division, which is retained and coexists with the new software-emulated one.

The following code snippet exemplifies the use of the new operator. As long as both a and b are positive and $b \neq 0$, the snippet will always print 1 when executed.

```
int a, b;

read(a);
read(b);
if (a / b == a [/] b) {
    write(1);
} else {
    write(0);
}
```

Tips: The simplest algorithm for computing division is called *repeated subtraction*. It involves subtracting the divisor from the dividend until the dividend becomes smaller than the divisor. The quotient is given by the number of subtractions performed.

In your solution you can write `INT_MAX` to refer to the constant $2^{31} - 1$.

1. Define the tokens (and the related declarations in **Acse.lex** and **Acse.y**). (1 points)
2. Define the syntactic rules or the modifications required to the existing ones. (2 points)
3. Define the semantic actions needed to implement the required functionality. (22 points)

4. Given the following Lance code snippet:

```
if (b) ; else { ; }
```

write down the syntactic tree generated during the parsing with the Bison grammar described in Acse.y *starting from the statement nonterminal*. (5 points)

5. (**Bonus**) Briefly explain in plain English words how to modify the solution given to the questions about the software-emulated division in order to add support for negative dividends and negative divisors.